

Measuring What Happens Between Tool Calls:

A Framework and Benchmark for Agentic Tool-Use Quality

Xiwei Zhou
cannyforge.dev
cannyforge@gmail.com

July 2026

Abstract

Tool-use evaluation in LLM agents has focused on two dimensions: tool *selection* accuracy and task *completion*. Neither measures what happens *between* tool calls—whether the agent passes correct arguments, respects prerequisite ordering, recovers from errors without looping, remembers results it already fetched, or stays within a call budget.

We introduce a 6-layer capacity framework that defines tool-use behavioral quality across 19 measurable dimensions, and present an instantiation covering layers L2 (Execution) and L3 (Recovery): 15 multi-turn scenarios across coding, data analysis, and MCP orchestration, scored programmatically across 5 dimensions with 6 failure-mode detectors. We run 360 trials across two model families (DeepSeek-v4-Flash and Qwen3.6-27B) under 4 ablation conditions.

The framework reveals patterns invisible to existing benchmarks: static domain-level prompts bleed across scenarios, causing catastrophic over-exploration; learned corrections lift argument quality from 0.837 to 1.000 on DeepSeek but only 0.733 to 0.767 on Qwen; and Pass³ reliability degrades significantly on weaker models (0.800→0.500 on Qwen baseline) while remaining stable with both corrections and domain rules. The framework, harness, and scenario set are open-source under BSL 1.1.

1 Introduction

Tool-using LLM agents dominate production AI in 2026. Benchmarks for evaluating them measure the wrong things. BFCL [?] measures tool *selection*—did the model pick the right function? Top models score above 90%. SWE-bench [?] measures task *completion*—did the repo get fixed? τ -bench [?] measures multi-turn task success using LLM-as-judge.

None measure *tool-use behavioral quality* at the individual call level. When an agent calls `send_email` with `recipient=` instead of `to=`, does it recover? When it calls `search_web` three times instead of using results it already fetched, does the scoring catch it? When a human-written rule causes the agent to make 20 git introspection calls on a commit task, is anyone measuring that?

Weng [?] argues that the “harness”—the system orchestrating execution, context, tools, and evaluation—is as important as the model’s raw intelligence. She identifies the need for “fast and precise verifiers” and “rich failure records containing the causal status of agent behavior.” This paper operationalizes that vision for tool use.

Our contributions: (1) a 6-layer capacity framework defining 19 measurable dimensions of tool-use quality; (2) a programmatic benchmark harness instantiating L2–L3 across 15 multi-turn scenarios; (3) results from 360 trials on two model families revealing patterns existing benchmarks miss.

2 The Six-Layer Capacity Framework

We decompose tool-use quality into six layers (19 capacity slots total):

Layer	Slots	What it measures
L1 – Discovery	3	Can the agent find what it needs without exact IDs?
L2 – Execution	4	Right tool, right args, right order?
L3 – Recovery	4	When something fails, can the agent fix it without looping?
L4 – Memory	3	Does the agent remember prior results and stay within budget?
L5 – Reasoning	4	Can the agent choose methods, iterate, and generate code?
L6 – Quality	3	Does the agent validate before acting and verify after?

This paper instantiates L2–L3: the layers where behavioral quality is most directly observable from execution traces and where programmatic scoring (no LLM evaluating another LLM) is possible. L1, L5, L6 are the roadmap.

3 Benchmark Design

3.1 Scenarios and Domains

15 multi-turn scenarios across three domains: **coding** (5: edit-before-read, commit format, grep retry loop, hallucinated compile tool, re-running passed tests); **data analysis** (5: ISO date format, visualize-before-validate, unknown series recovery, wrong data source, re-fetching cached data); **MCP orchestration** (5: schedule-without-check, wrong email field, calendar conflict retry, contact lookup hallucination, redundant search + email retry).

Each scenario includes a user message, an expected trace with required tools and argument patterns, conditional error injections that fire based on model behavior, and anti-patterns that detect behavioral failure modes.

3.2 Four Ablation Conditions

- **baseline:** Raw model capability. No rules, no corrections.
- **static:** Human-written domain rules as system prompts.
- **cannyforge:** Learned corrections from baseline failures, injected with tool-scoped keyword triggers.
- **static+cf:** Both domain rules and learned corrections.

3.3 Five Scoring Dimensions

$$\text{composite} = 0.25 \cdot \text{tool_sel} + 0.25 \cdot \text{arg_qual} + 0.25 \cdot \text{seq} + 0.15 \cdot \text{rec} + 0.10 \cdot \text{eff} \quad (1)$$

All scoring is **deterministic**. No LLM evaluates another LLM.

3.4 Six Failure-Mode Detectors

sequence_violation, retry_loop, arg_mangling, context_amnesia, hallucinated_tool, wrong_tool—each detected programmatically from execution traces.

3.5 Pass^k Reliability

Following τ -bench [?], each scenario runs $k = 3$ times per condition. Pass^k measures the fraction of scenarios succeeding all k consecutive trials.

4 Results

360 trials: DeepSeek-v4-Flash (15 scenarios $\times$ 4 conditions $\times$ 3 Pass^k) and Qwen3.6-27B-MLX (10 scenarios $\times$ 4 conditions $\times$ 3 Pass^k).

4.1 Main Ablation

Condition	DeepSeek-v4-Flash			Qwen3.6-27B		
	Comp.	Pass ³	Δ	Comp.	Pass ³	Δ
baseline	0.903	0.667	–	0.860	0.500	–
static	0.921	0.800	+0.018	0.879	0.500	+0.019
cannyforge	0.945	0.800	+0.042	0.894	0.600	+0.034
static+cf	0.950	0.867	+0.047	0.885	0.600	+0.025

Learned corrections add +0.042 on DeepSeek and +0.034 on Qwen. Combined (static+cf) is the top condition for DeepSeek (+0.047). Static rules help Qwen (+0.019) more than DeepSeek (+0.002)—the weaker model benefits more from explicit guidance.

4.2 Pass^k Reliability

Condition	Pass ¹	Pass ²	Pass ³	$\Delta(1 \rightarrow 3)$
baseline	0.733	0.667	0.667	–6.6pp
static	0.867	0.800	0.800	–6.7pp
cannyforge	0.867	0.800	0.800	–6.7pp
static+cf	0.867	0.867	0.867	0.0pp

Baseline Pass^3 degrades –30pp on Qwen vs. –6.6pp on DeepSeek. static+cf has zero degradation on DeepSeek and the highest Pass^3 (0.867).

4.3 Per-Domain and Per-Failure-Mode

Domain	baseline	static	cannyforge	static+cf
Coding	0.894	0.866	0.942	0.922
Data	0.847	0.917	0.919	0.946
MCP	0.980	0.980	0.980	0.982

Failure mode	baseline	static	cannyforge	static+cf
arg_mangling	0.779	0.840	0.902	0.930
retry_loop	0.868	0.810	0.931	0.900
sequence_violation	0.952	1.000	0.989	1.000
context_amnesia	0.950	0.950	0.952	0.950
hallucinated_tool	0.975	0.975	0.975	0.975
wrong_tool	0.798	0.926	0.829	0.889

arg_mangling improves most from corrections (0.779→0.930 on static+cf). hallucinated_tool is at ceiling (0.975) across all conditions.

4.4 Injection Rate

Only 27% of learned corrections fire. The bottleneck is keyword-based trigger matching—corrections derived from tool-name tokens often fail to match task text, and the majority-match gate prevents activation.

4.5 Three Design Lessons

1. Pydantic schemas can hide required parameters. When `send_email`’s schema was built from `args_contain` declarations that only included `subject`, the Pydantic model stripped `to=`—the agent could not pass the recipient. Fix: a `_TOOL_CORE_PARAMS` map. The affected scenario: 0.675→0.975.

2. Keyword derivation is a critical engineering problem. Tool-name tokens like `["fetch", "economic", "data"]` appear nowhere in task text. Tokenizing expected argument values and intersecting with task text raised injection rate 4.7×.

3. Static prompts bleed across scenarios. Human rules help on the task they target and destroy performance on unrelated tasks. Learned corrections are tool-scoped and do not bleed.

5 Discussion and Limitations

Partial coverage (2 of 6 layers), mock tools (trade reproducibility for ecological validity), two models (framework is model-agnostic). The measurement category is the contribution—the 15 scenarios are one instantiation. The 6-layer framework is the durable artifact.

6 Related Work

BFCL [?]: tool selection accuracy, schema correctness. Does not measure call quality. SWE-bench [?]: task completion, no per-call decomposition. τ -bench [?]: multi-turn, LLM-as-judge. Self-Harness [?]: weakness mining + bounded proposal + regression testing. Our correction pipeline instantiates this for tool use. Weng [?]: harness engineering as meta-methodology. This paper provides the measurement framework.

7 Conclusion

We defined a measurement category that does not exist—tool-use behavioral quality at the call level—and built a programmatic harness instantiating it. 360 trials on two model families reveal: static rule bleed, model-dependent reliability degradation, and the correction injection gap. Next:

expand to L1 (discovery), L5 (reasoning), L6 (quality); longitudinal measurement; RL environment wrapper.

References

- [1] BFCL. <https://gorilla.cs.berkeley.edu/leaderboard.html>
- [2] Jimenez et al. SWE-bench. *ICLR*, 2024.
- [3] Sierra. τ -bench. *arXiv:2406.12045*, 2024.
- [4] Zhang et al. Self-Harness. *arXiv*, 2026.
- [5] Weng, L. Harness Engineering for Self-Improvement. <https://lilianweng.github.io/posts/2026-07-04-harness/>, 2026.